

Séance 4 — Visualiser et acquérir des mesures

Introduction à la programmation Python — IUT 1re année

Séance 4

Visualiser et acquérir des mesures

Matplotlib, instrumentation, liaisons série et réseau

Une série de nombres devient plus facile à interpréter lorsqu'elle est représentée graphiquement. Python peut aussi recevoir directement les mesures d'un instrument.

À la fin de la séance, vous devez être capables de produire un graphique scientifique lisible et d'expliquer les étapes d'un dialogue avec un instrument.

Cette séance termine la chaîne commencée avec les fichiers et NumPy. Matplotlib transforme les données en représentations visuelles ; l'instrumentation et la liaison série montrent comment ces données peuvent provenir du monde physique.

Rôle de la visualisation

Un indicateur résume les données ; un graphique montre leur organisation.

Une représentation peut révéler :

- une tendance ou une évolution temporelle ;
- une relation entre deux grandeurs ;
- une dispersion ou une valeur aberrante ;
- un changement brutal ou un dépassement de seuil.

Le graphique doit être choisi en fonction de la question scientifique, pas seulement pour son apparence.

Choisir une représentation

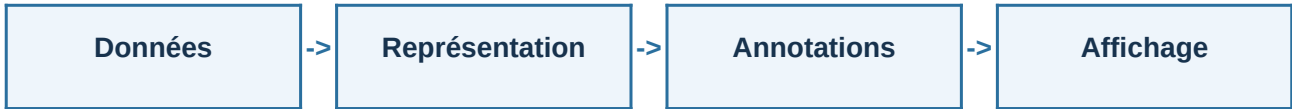
Question	Représentation adaptée
Comment une grandeur évolue-t-elle ?	courbe
Deux variables sont-elles liées ?	nuage de points
Comment les valeurs sont-elles réparties ?	histogramme

Une même série peut être représentée de plusieurs manières, mais chaque graphique met en évidence une information différente.

Relier des points par une courbe suggère un ordre, souvent temporel. Un nuage de points ne suppose pas cette continuité et sert à étudier une relation. Un histogramme regroupe les valeurs par intervalles ; il ne doit pas être confondu avec un diagramme en barres représentant des catégories.

Construire un graphique avec Matplotlib

```
import matplotlib.pyplot as plt temps = [0, 1, 2, 3] temperatures = [20.1, 20.4, 20.9, 21.3] plt.plot(temps, temperatures) plt.xlabel("Temps (s)") plt.ylabel("Température (°C)") plt.title("Évolution de la température") plt.show()
```



Le module `matplotlib.pyplot`, généralement importé sous l'alias `plt`, fournit une interface simple pour construire une figure. Les deux séries transmises à `plot` doivent avoir le même nombre d'éléments : chaque abscisse correspond à une ordonnée.

Représenter une évolution

```
import matplotlib.pyplot as plt temps = [0, 1, 2, 3] temperatures = [20.1, 20.4, 20.9, 21.3] plt.plot( temps, temperatures, color="tab:blue", marker="o", label="Température", )
```

	Argument	Rôle
<code>color</code>		couleur de la courbe
<code>marker</code>		symbole placé sur les mesures
<code>label</code>		nom utilisé dans la légende

Une ligne est pertinente lorsque l'ordre des points possède un sens.

Nuage de points et histogramme

```
import matplotlib.pyplot as plt tensions = [1.0, 2.0, 3.0, 4.0] courants = [0.10, 0.19, 0.31, 0.40] plt.scatter(tensions, courants)
```

Le nuage de points montre la relation entre deux variables mesurées sur les mêmes observations.

```
temperatures = [20.1, 20.4, 20.9, 21.3, 20.8, 20.5] plt.hist(temperatures, bins=10)
```

L'histogramme découpe l'étendue des valeurs en intervalles et compte les observations dans chacun d'eux. Le choix de `bins` influence la lecture de la distribution.

Annoter un graphique scientifique

Un graphique doit pouvoir être compris sans l'explication orale qui l'accompagne.

```
plt.xlabel("Temps (s)") plt.ylabel("Température (°C)") plt.title("Évolution de la température") plt.grid(alpha=0.3) plt.legend()
```

Les axes indiquent la grandeur et son unité. Une légende est nécessaire lorsque plusieurs séries sont tracées ou lorsqu'un `label` doit être identifié.

Un titre décrit la situation représentée sans répéter simplement les axes. Une grille légère facilite la lecture des valeurs mais ne doit pas dominer les données. `legend()` utilise les textes fournis avec l'argument `label` des tracés.

Figure, axes et sous-graphiques

Matplotlib distingue la **figure**, qui contient l'ensemble du document, et les **axes**, qui sont les zones de tracé.

```
import matplotlib.pyplot as plt temps = [0, 1, 2, 3] temperatures = [20.1, 20.4, 20.9, 21.3] fig, axes = plt.subplots(1, 2, figsize=(10, 4)) axes[0].plot(temps, temperatures) axes[1].hist(temperatures, bins=10) fig.tight_layout()
```

Les sous-graphiques permettent de comparer plusieurs représentations sans les superposer.

Malgré son nom, un objet `Axes` représente une zone complète de tracé avec ses deux axes gradués. L'interface orientée objet (`fig, axes`) devient plus claire que `plt` dès que la figure contient plusieurs graphiques.

Exporter une figure

Une figure peut être enregistrée pour être intégrée dans un rapport. Cet exemple complet crée la figure avant de l'enregistrer :

```
import matplotlib.pyplot as plt fig, ax = plt.subplots() ax.plot([0, 1, 2, 3], [20.1, 20.4, 20.9, 21.3]) ax.set_xlabel("Temps (s)") ax.set_ylabel("Température (°C)") fig.savefig("temperature.png", dpi=300, bbox_inches="tight", )
```

`dpi` contrôle la résolution d'une image matricielle ; `bbox_inches="tight"` limite les marges inutiles.

La sauvegarde est généralement effectuée avant `plt.show()`.

Le format PNG convient aux documents courants. Les formats vectoriels comme SVG ou PDF restent nets lors d'un agrandissement. Avec l'interface `pyplot`, la fonction équivalente est `plt.savefig()`.

Chaîne d'instrumentation

Une mesure numérique résulte de plusieurs transformations :

Grandeur physique

v

Capteur

v

Électronique d'acquisition

v

Communication

v

Ordinateur et programme Python

v

Analyse et représentation

Python intervient après la conversion de la grandeur physique en données transmissibles.

Le capteur réagit à une grandeur telle qu'une température ou une tension. L'électronique conditionne et numérise le signal. Le système communique ensuite les valeurs à l'ordinateur, où Python peut automatiser leur stockage, leur analyse et leur représentation.

Liaison matérielle et liaison logicielle

Pour dialoguer avec un instrument, il faut d'abord savoir comment s'y brancher, puis quelles instructions lui envoyer :

Élément	Question associée	Exemples
liaison matérielle	quel câble et quel connecteur ?	USB, série, Ethernet, GPIB
liaison logicielle	quelles règles et commandes ?	SCPI, VISA, Modbus, protocole constructeur

Deux appareils équipés du même connecteur USB ou Ethernet ne parlent pas forcément le même langage.

La liaison matérielle décrit le transport. La liaison logicielle définit la structure, l'ordre et la signification des messages. Avant tout achat ou essai, il faut donc vérifier les deux dans la documentation technique.

Commandes SCPI

SCPI (*Standard Commands for Programmable Instruments*) définit une syntaxe textuelle utilisée par de nombreux instruments de laboratoire.

```
*IDN? MEAS:TEMP? TEMP:UNIT DEG
```

- `*IDN?` demande l'identification de l'instrument ;
- `MEAS:TEMP?` demande une mesure de température ;
- `TEMP:UNIT DEG` règle un paramètre sans attendre nécessairement de réponse ;
- le point d'interrogation signale une requête qui attend une réponse.

Dans tous les cas, seule la documentation de l'instrument fait foi.

SCPI standardise une partie du vocabulaire, mais les commandes réellement prises en charge dépendent de l'instrument. Sa documentation reste la référence. Les commandes sont généralement terminées par un caractère de fin de ligne.

Paramètres d'une liaison série

Une liaison série transmet une suite d'octets. Les deux équipements doivent utiliser des paramètres compatibles.

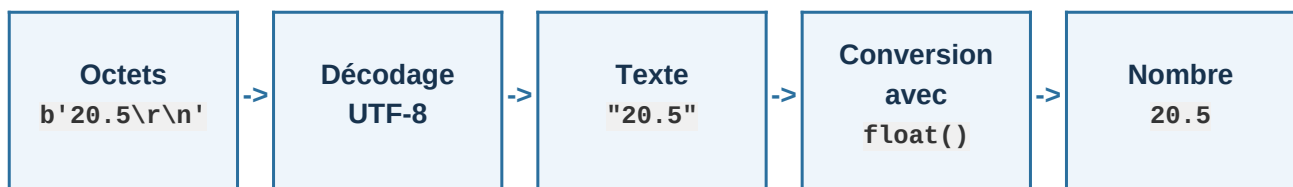
Paramètre	Rôle
port	identifier l'interface, par exemple <code>COM3</code>
baudrate	fixer la vitesse de transmission
format	fixer le nombre de bits, la parité et les bits d'arrêt
timeout	limiter la durée d'attente d'une lecture
fin de ligne	délimiter les messages

Sous Linux, un port peut notamment apparaître sous la forme `/dev/ttyUSB0` ou `/dev/ttyACM0`.

Sous Windows, un port série ne peut généralement être ouvert que par une application à la fois. Le nom d'un adaptateur USB-série peut aussi changer après un débranchement.

Octets, texte et nombre

Le port série échange des `bytes`, alors que le programme veut généralement manipuler du texte puis un nombre.



```
ligne = b"20.5\r\n" # exemple de réponse reçue
texte = ligne.decode("utf-8").strip()
mesure = float(texte)
```

L'encodage doit être identique du côté de l'émetteur et du récepteur.

Un octet est une valeur numérique comprise entre 0 et 255. Un encodage comme UTF-8 définit comment une suite d'octets représente des caractères. `strip()` retire ici les caractères de fin de ligne avant la conversion numérique.

Lire avec pyserial

```
import serial with serial.Serial("COM3", baudrate=19200, timeout=1) as port: ligne = port.readline() texte = ligne.decode("utf-8").strip() mesure = float(texte)
```

Le bloc `with` ferme automatiquement le port. `readline()` lit jusqu'à une fin de ligne ou jusqu'à l'expiration du délai.

L'ouverture peut échouer si le port n'existe pas, est déjà utilisé ou si les droits sont insuffisants. Une lecture peut être vide après un timeout et une réponse peut être invalide. Une application réelle doit donc vérifier les données reçues avant leur conversion.

Envoyer une commande

Une commande textuelle doit être encodée avant son envoi :

```
import serial with serial.Serial("COM3", baudrate=19200, timeout=1) as port: commande = "*IDN?\n".encode("utf-8") port.write(commande) reponse = port.readline().decode("utf-8").strip()
```

Commande textuelle

v

Encodage en octets

v

Envoi et réception

v

Décodage des octets

v

Réponse textuelle

La terminaison `\n` est ici incluse dans la commande car de nombreux instruments attendent une fin de ligne avant de traiter le message. La terminaison exacte doit être vérifiée dans la documentation de l'appareil.

Communiquer par le réseau

Un instrument Ethernet est généralement identifié par deux informations :

Information	Rôle	Exemple
adresse IP	identifier l'instrument sur le réseau	192.168.0.26
port TCP	identifier le service qui reçoit les commandes	8000

Le dialogue suit ensuite les mêmes étapes qu'avec une liaison série :



Les valeurs à utiliser et le protocole de dialogue sont indiqués dans la documentation de l'instrument. Une prise Ethernet ne garantit pas à elle seule que l'appareil accepte des commandes SCPI par une connexion TCP directe.

Python fournit le module `socket` pour les communications réseau de bas niveau. Les constantes et méthodes nécessaires dépendent du protocole retenu ; elles ne sont pas à mémoriser dans ce cours. En pratique, une bibliothèque constructeur ou VISA peut fournir une interface plus simple. On commencera donc toujours par la documentation de l'instrument plutôt que par un exemple de `socket` générique.

Acquisition avec une carte Arduino

Une carte Arduino peut envoyer périodiquement une mesure terminée par un retour à la ligne. Python peut alors :

1. ouvrir la liaison série ;
2. lire et convertir chaque mesure ;
3. stocker les valeurs ;
4. calculer des indicateurs avec NumPy ;
5. tracer et sauvegarder un graphique.



v

Python

v

NumPy

v

Matplotlib

Une source simulée permet de tester le programme lorsqu'aucun matériel n'est disponible.

Synthèse du cours

Les quatre séances construisent une chaîne complète :

Étape

programmer
stocker
calculer
visualiser
acquérir

Outils principaux

variables, conditions, fonctions, boucles
listes, fichiers, CSV
NumPy et statistiques descriptives
Matplotlib
instrument, liaisons série ou réseau, Arduino

L'objectif est de transformer une grandeur physique en données interprétables, puis en résultat scientifique communicable.

Chaque étape possède une responsabilité distincte. Cette séparation permet de tester les traitements avec des données simulées avant de connecter le matériel, puis de remplacer la simulation par une acquisition réelle sans reconstruire tout le programme.

Exercices — Séance 4 : Visualiser et acquérir des mesures

Fichier nécessaire

Copier `mesures.csv` depuis le dossier `seance3` vers le dossier `seance4`, puis enregistrer les programmes de cette feuille dans `seance4`.

Ex. 1 — Première courbe À partir du CSV de la séance 3, tracer température en fonction du temps avec `plt.plot`, en ajoutant titre et légendes des axes.

Ex. 2 — Nuage de points et histogramme Dans un même script, créer d'abord une figure contenant un nuage de points (deux séries de mesures liées), puis une seconde figure contenant l'histogramme d'une série de valeurs.

Ex. 3 — Personnalisation et export Reprendre l'Ex. 1 : changer couleur/style de ligne, ajouter une grille et une légende, puis exporter le graphique en `.png`.

Ex. 4 — Détection de dépassement de seuil Ajouter au graphique une ligne horizontale représentant un seuil (`plt.axhline`), et faire apparaître en rouge les points qui le dépassent.

Ex. 5 — Sous-figures Créer une figure avec 2 sous-graphiques côte à côte (courbe + histogramme) pour la même série de mesures, avec `plt.subplots`.

Ex. 6 — Lecture d'un port série (si matériel disponible) Avec `pyserial`, lister les ports avec `serial.tools.list_ports.comports()`, se connecter à une carte, lire 10 mesures, les stocker dans une liste puis les tracer. Sans matériel, relire progressivement les lignes de `data/donnees/mesures.csv` comme si elles arrivaient du port série.